

3장 SQL 튜닝

깃헙에서 SQL 실행문 실행

깃헙 링크

<https://github.com/7ieon/SQLtune/tags>

- table install

```
PS C:\SQLtune-main> Start-Process -FilePath "C:\SQLtune-main\mysql\bin\mysql.exe" -ArgumentList "-u root -p00000" -NoNewWindow -Wait -RedirectStandardOutput "C:\SQLtune-main\mysql\bin\mysql.log"
mysql: [Warning] Using a password on the command line interface can be insecure.
INFO
LOADING emp.sql
INFO
LOADING dept.sql
INFO
LOADING emp_hist1.sql
INFO
LOADING emp_hist2.sql
INFO
LOADING grade.sql
INFO
LOADING sal1.sql
INFO
LOADING sal2.sql
INFO
LOADING sal3.sql
INFO
LOADING sal4.sql
INFO
LOADING sal5.sql
INFO
LOADING sal6.sql
PS C:\SQLtune-main>
```

- show table

```
mysql> use tuning;
Database changed
mysql> show tables;
+-----+
| Tables_in_tuning |
+-----+
| 급여              |
| 부서              |
| 부서관리자        |
| 부서사원_매핑     |
| 사원              |
| 사원출입기록      |
| 직급              |
+-----+
7 rows in set (0.03 sec)
```

SQL 실행 계획 확인

- **EXPLAIN**, **DESCRIBE**, **DESC** 사용

```
mysql> EXPLAIN SELECT COUNT(1) FROM 사원;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | 사원 | NULL | index | NULL | I_입사일자 | 3 | NULL | 297385 | 100.00 | Using index |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
```

- EXPLAIN키워드를 입력하면, 옵티마이저가 만든 실행계획이 출력된다,

```
mysql> EXPLAIN
-> SELECT *
-> FROM 사원
-> WHERE 사원번호 BETWEEN 100001 AND 20000;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | 사원 | NULL | index | NULL | NULL | NULL | NULL | NULL | NULL | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

3.2.2 기본 실행 계획 항목 분석

- SQL문이 비효율적으로 수행되는지, 아닌지를 알기 위해서는 각 항목의 의미를 알아야 한다. 이를 통해 튜닝의 방향성을 잡을 수 있기 때문!

id

- 실행 순서를 표시하는 숫자, SQL문이 수행되는 차례를 ID로 표기한 것
 - 조인할 때는 똑같은 ID가 표시된다.
 - ID숫자가 작을수록 먼저 수행된 것, ID가 같은 값이라면 두개의 테이블이 조인된 것

sql문

```
mysql> EXPLAIN
```

```
-> SELECT 사원.사원번호, 사원.이름, 사원.성, 급여.연봉,  
-> (SELECT MAX(부서번호)  
-> FROM 부서사원_매핑 AS 매핑 WHERE 매핑.사원번호 = 사원.사원번호)  
-> FROM 사원, 급여;
```

```
mysql> EXPLAIN
-> SELECT 사원.사원번호, 사원.이름, 사원.성, 급여.연봉,
-> (SELECT MAX(부서번호)
-> FROM 부서사원_매핑 AS 매핑 WHERE 매핑.사원번호 = 사원.사원번호) 카운트
-> FROM 사원, 급여;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	사원	NULL	ALL	NULL	NULL	NULL	NULL	297385	100.00	NULL
1	PRIMARY	급여	NULL	ALL	NULL	NULL	NULL	NULL	2836398	100.00	Using join buffer (hash join)
2	DEPENDENT SUBQUERY	매핑	NULL	ref	PRIMARY	PRIMARY	4	tuning.사원.사원번호	1	100.00	Using index

3 rows in set, 2 warnings (0.00 sec)

id	select_type	table	partitions	type	possible_keys	key
1	PRIMARY	사원	NULL	const	PRIMARY	PRIMARY
1	PRIMARY	급여	NULL	ref	PRIMARY	PRIMARY
2	DEPENDENT SUBQUERY	NULL	NULL	NULL	NULL	NULL

key_len	ref	rows	filtered	Extra
4	const	1	100.00	NULL
4	const	17	100.00	NULL
NULL	NULL	NULL	NULL	Select tables optimized away

3 rows in set, 2 warnings (0.00 sec)

select_type

- sql문을 구성하는 select문의 유형을 출력하는 항목
- 단순히 from절에 위치한 것인지, 서브쿼리인지 Union으로 묶인 select절인지 정보를 제공

select_type - SIMPLE

```
mysql> explain
-> select * from 사원 where 사원번호 = 100000;
```

- UNION이나 내부쿼리가 없는 SELECT문이라는 걸 의미하는 유형, 단순한 SELECT 구문으로만 작성된 경우,

```
mysql> EXPLAIN
      -> SELECT * FROM 사원 WHERE 사원번호 = 100000;
```

id	select_type	table	partitions	type	possible_keys	key
1	SIMPLE	사원	NULL	const	PRIMARY	PRIMARY

key_len	ref	rows	filtered	Extra

```
mysql> explain
      -> select * from 사원 where 사원번호 = 100000;
```

id	select_type	table	partitions	type	possible_keys	key	key_len
1	SIMPLE	사원	NULL	const	PRIMARY	PRIMARY	4

1 row in set, 1 warning (0.00 sec)

- 예 2

```
explain
select 사원.사원번호, 사원.이름, 사원.성, 급여.연봉
from 사원,
      (select 사원번호, 연봉
       from 급여
       where 연봉 > 80000) as 급여
where 사원.사원번호 = 급여.사원번호
and 사원.사원번호 between 10001 and 10010;
```

id	select_type	table	partitions	type	possible_keys	key
1	PRIMARY	사원	NULL	const	PRIMARY	PRIMARY
1	PRIMARY	급여	NULL	ref	PRIMARY	PRIMARY
2	DEPENDENT SUBQUERY	NULL	NULL	NULL	NULL	NULL

key_len	ref	rows	filtered	Extra
4	const	1	100.00	NULL
4	const	17	100.00	NULL
NULL	NULL	NULL	NULL	Select tables optimized away

3 rows in set, 2 warnings (0.00 sec)

```
mysql> explain
-> select 사원.사원번호, 사원.이름, 사원.성, 급여.연봉
-> from 사원
-> (select 사원번호, 연봉
-> from 급여
-> where 연봉 > 80000) as 급여
-> where 사원.사원번호 = 급여.사원번호
-> and 사원.사원번호 between 10001 and 10010;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	사원	NULL	range	PRIMARY	PRIMARY	4	NULL	10	100.00	Using where
1	SIMPLE	급여	NULL	ref	PRIMARY	PRIMARY	4	tuning.사원.사원번호	9	33.33	Using where

2 rows in set, 1 warning (0.01 sec)

selct_type - PRIMARY

- 서브쿼리가 포함된 SQL문이 있을 때 첫 번째 select문에 해당하는 구문에 표시되는 유형,
 - 서브쿼리를 감싸는 외부쿼리거나, UNION이 포함된 SQL문에서 첫 번째로 select키워드가 작성된 구문에 표시됨

EXPLAIN

```
SELECT 사원.사원번호, 사원.이름, 사원.성,
       (SELECT MAX(부서번호)
        FROM 부서사원_매핑 AS 매핑 WHERE 매핑.사원번호 = 사원.사원번호)
FROM 사원
WHERE 사원.사원번호 = 10001;
```

```
-> FROM 사원
-> WHERE 사원.사원번호 = 100001;
```

id	select_type	table	partitions	type	possible_keys
1	PRIMARY	사원	NULL	const	PRIMARY
2	SUBQUERY	매핑	NULL	index	NULL

key	key_len	ref	rows	filtered	Extra
PRIMARY	4	const	1	100.00	NULL
인덱스_부서번호	16	NULL	331143	100.00	Using index

2 rows in set (0.00 sec)

```
mysql> EXPLAIN
-> SELECT 사원.사원번호, 사원.이름, 사원.성,
-> (SELECT MAX(부서번호)
-> FROM 부서사원_매핑 AS 매핑 WHERE 매핑.사원번호 = 사원.사원번호) 카운트
-> FROM 사원
-> WHERE 사원.사원번호 = 10001;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	사원	NULL	const	PRIMARY	PRIMARY	4	const	1	100.00	NULL
2	DEPENDENT SUBQUERY	사원	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	Select tables optimized away

2 rows in set, 2 warnings (0.00 sec)

- 또는 UNION ALL구문으로 통합된 SQL문에서 처음 SELECT 구문이 작성된 쿼리가 먼저 접근한다는 의미로 PRIMARY가 출력됨

explain

```
select 사원1.사원번호, 사원1.이름, 사원1.성
  from 사원 as 사원1
 where 사원1.사원번호 = 100001
```

union all

```
select 사원2.사원번호, 사원2.이름, 사원2.성
```

```

from 사원 as 사원2
where 사원2.사원번호 = 100002;

```

id	select_type	table	partitions	type	possible_keys	key
1	PRIMARY	사원1	NULL	const	PRIMARY	PRIMARY
2	UNION	사원2	NULL	const	PRIMARY	PRIMARY

key_len	ref	rows	filtered	Extra
4	const	1	100.00	NULL
4	const	1	100.00	NULL

2 rows in set (0.00 sec)

```

mysql> explain
-> select 사원1.사원번호, 사원1.이름, 사원1.성
-> from 사원 as 사원1
-> where 사원1.사원번호 = 100001
-> union all
-> select 사원2.사원번호, 사원2.이름, 사원2.성
-> from 사원 as 사원2
-> where 사원2.사원번호 = 100002;

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	사원1	NULL	const	PRIMARY	PRIMARY	4	const	1	100.00	NULL
2	UNION	사원2	NULL	const	PRIMARY	PRIMARY	4	const	1	100.00	NULL

2 rows in set, 1 warning (0.00 sec)

selct_type - SUBQUERY

- 독립적으로 수행되는 서브쿼리,
- SELECT절의 서브쿼리나, WHERE절의 중첩 서브쿼리일 경우, 해당

```

explain
select (select count(*)
       from 부서사원_매핑 as 매핑
       )as 카운트,

       (select max(연봉)
        from 급여
        ) as 급여;

```

id	select_type	table	partitions	type	possible_keys	key
1	PRIMARY	NULL	NULL	NULL	NULL	NULL
3	SUBQUERY	급여	NULL	ALL	NULL	NULL
2	SUBQUERY	매핑	NULL	index	NULL	I_부서번호

key_len	ref	rows	filtered	Extra
NULL	NULL	NULL	NULL	No tables used
NULL	NULL	2838731	100.00	NULL
12	NULL	331143	100.00	Using index

3 rows in set, 1 warning (0.00 sec)

```

mysql> explain
-> select (select count(*)
-> from 부서사원_매핑 as 매핑
-> )as 카운트,
->
-> (select max(연봉)
-> from 급여
-> ) as 급여;

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	No tables used
3	SUBQUERY	급여	NULL	ALL	NULL	NULL	NULL	NULL	2838398	100.00	NULL
2	SUBQUERY	매핑	NULL	index	NULL	I_부서번호	12	NULL	331143	100.00	Using index

3 rows in set, 1 warning (0.00 sec)

selct_type - DERIVED

- from 절에 작성된 서브쿼리,

- from절의 별도 임시 테이블인 인라인 뷰를 의미

explain

SELECT 사원.사원번호, 급여.연봉

FROM 사원

JOIN (

SELECT 사원번호, MAX(연봉) AS 연봉

FROM 급여

WHERE 사원번호 BETWEEN 10001 AND 20000

GROUP BY 사원번호

) AS 급여

ON 사원.사원번호 = 급여.사원번호;

id	select_type	table	partitions	type	possible_keys	key
1	PRIMARY	<derived2>	NULL	system	NULL	NULL
1	PRIMARY	사원	NULL	const	PRIMARY	PRIMARY
2	DERIVED	급여	NULL	range	PRIMARY	PRIMARY

key_len	ref	rows	filtered	Extra
NULL	NULL	1	100.00	NULL
4	const	1	100.00	Using index
4	NULL	184756	100.00	Using where

3 rows in set (0.04 sec)

```
mysql> explain
-> SELECT 사원.사원번호, 급여.연봉
-> FROM 사원
-> JOIN (
->   SELECT 사원번호, MAX(연봉) AS 연봉
->   FROM 급여
->   WHERE 사원번호 BETWEEN 10001 AND 20000
->   GROUP BY 사원번호
-> ) AS 급여
-> ON 사원.사원번호 = 급여.사원번호;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL	184756	100.00	NULL
1	PRIMARY	사원	NULL	eq_ref	PRIMARY	PRIMARY	4	급여.사원번호	1	100.00	Using index
2	DERIVED	급여	NULL	range	PRIMARY, I_사용여부	PRIMARY	4	NULL	184756	100.00	Using where

3 rows in set, 1 warning (0.01 sec)

selct_type - UNION RESULT

- UNION ALL이 아닌 UNION으로 SELECT절을 결합했을 때 출력
 - UNION은 출력 결과에 중복이 없음, 따라서 SELECT절에서 데이터를 가져와 정렬하여 중복체크하는 과정이 필요
 - 따라서 UNION RESULT는 별도의 메모리 또는 디스크에 임시 테이블을 만들어 중복을 제거하겠다는 의미

```
explain
select 사원_통합.*
  from (
    select max(입사일자) as 입사일자
      from 사원 as 사원1
     where 성별 = 'M'
    union
    select min(입사일자) as 입사일자
      from 사원 as 사원2
     where 성별 = 'M'
  ) as 사원_통합;
```

id	select_type	table	partitions	type	possible_keys
1	PRIMARY	<derived2>	NULL	ALL	NULL
2	DERIVED	사원1	NULL	ref	I_성별_성

3	UNION	사원2	NULL	ref	I_성별_성	
	UNION RESULT	<union2,3>	NULL	ALL	NULL	

key	key_len	ref	rows	filtered	Extra	

NULL	NULL	NULL	2	100.00	NULL	
I_성별_성	1	const	149578	100.00	NULL	
I_성별_성	1	const	149578	100.00	NULL	
NULL	NULL	NULL	NULL	NULL	Using temporary	

4 rows in set, 1 warning (0.00 sec)						

```
mysql> explain
-> select 사원_통합.*
-> from (
-> select max(임사일자) as 임사일자
-> from 사원 as 사원1
-> where 성별 = 'M'
-> union
-> select min(임사일자) as 임사일자
-> from 사원 as 사원2
-> where 성별 = 'M'
-> ) as 사원_통합;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL	2	100.00	NULL
2	DERIVED	사원1	NULL	ref	I_성별_성	I_성별_성	1	const	148692	100.00	NULL
3	UNION	사원2	NULL	ref	I_성별_성	I_성별_성	1	const	148692	100.00	NULL
4	UNION RESULT	<union2,3>	NULL	ALL	NULL	NULL	NULL	NULL	NULL	NULL	Using temporary

4 rows in set, 1 warning (0.01 sec)

selct_type - DEPENDENT SUBQUERY

- UNION, UNION ALL을 사용하는 서브쿼리가, 메인 테이블의 영향을 받는 경우,
- UNION으로 연결된 단위 쿼리들 중에서 처음으로 작성한 단일쿼리에 해당하는 경우
 - ⇒ 즉 UNION으로 연결되는 첫 번째 단위 쿼리가 독립적으로 수행하지 못하고 메인 테이블로부터 값을 하나씩 공급받는 구조 (AND 사원1.사원번호 = 관리자.사원번호) 이므로 성능적으로 불리하여 SQL문이 튜닝 대상이 된다

```
explain
select 관리자.부서번호,
```

```

(select 사원1.이름
  from 사원 as 사원1
 where 성별 = 'F'
        and 사원1.사원번호 = 관리자.사원번호

 union all

 select 사원2.이름
        from 사원 as 사원2
        where 성별 = 'M'
        and 사원2.사원번호 = 관리자.사원번호
) as 이름
from 부서관리자 as 관리자;

```

```

+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY          | 관리자 | NULL | index | NULL |
| 2 | DEPENDENT SUBQUERY | 사원1  | NULL | eq_ref | PRIMARY,I_성별_성 |
| 3 | DEPENDENT UNION   | 사원2  | NULL | eq_ref | PRIMARY,I_성별_성 |
+-----+-----+-----+-----+-----+-----+

key          | key_len | Extra |
+-----+-----+-----+
I_부서번호  | 12      | Using index |
PRIMARY     | 4       | Using where |
PRIMARY     | 4       | Using where |
+-----+-----+-----+
3 rows in set, 3 warnings (0.00 sec)

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	관리자	NULL	index	NULL	I_부서번호	12	NULL	24	100.00	Using index
2	DEPENDENT SUBQUERY	사원1	NULL	eq_ref	PRIMARY,I_성별_성	PRIMARY	4	tuning.관리자.사원번호	1	50.00	Using where
3	DEPENDENT UNION	사원2	NULL	eq_ref	PRIMARY,I_성별_성	PRIMARY	4	tuning.관리자.사원번호	1	50.00	Using where

selct_type - DEPENDENT UNION

- UNION, UNION, UNION ALL을 사용하는 서브쿼리가, 메인 테이블의 영향을 받는 경우,
- UNION으로 연결된 단위 쿼리들 중에서 두번째 단일쿼리에 해당하는 경우

```

explain
select 관리자.부서번호,
       (select 사원1.이름
        from 사원 as 사원1
        where 성별 = 'F'
          and 사원1.사원번호 = 관리자.사원번호

       union all

       select 사원2.이름
        from 사원 as 사원2
        where 성별 = 'M'
          and 사원2.사원번호 = 관리자.사원번호
       ) as 이름
from   부서관리자 as 관리자;

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	관리자	NULL	index	NULL	1. 부서번호	12	NULL	24	100.00	Using index
2	DEPENDENT SUBQUERY	사원1	NULL	eq_ref	PRIMARY, 1.성별_성	PRIMARY	4	tuning. 관리자. 사원번호	1	50.00	Using where
3	DEPENDENT UNION	사원2	NULL	eq_ref	PRIMARY, 1.성별_성	PRIMARY	4	tuning. 관리자. 사원번호	1	50.00	Using where

select_type - UNCACHEABLE SUBQUERY

- 메모리에 상주하여 재사용되어야 할 서브쿼리가 재사용되지 못할 때 출력되는 유형
 - 해당 서브쿼리 안에 사용자 정의 함수나, 사용자 변수가 포함되거나
 - RAND(), UUID() 함수 등을 사용하여 매번 조회시 마다 결과가 달라지는 경우
- 만약 자주호출되는 SQL문이라면 메모리에 서브쿼리 결과가 상주할 수 있도록 변경하는 방향으로 SQL성능 튜닝을 검토해볼 수 있다.
- 0-1사이의 소수점을 기준으로 17자리 숫자를 출력하는 함수
 - 매번 출력값이 달라지는 RAND()때문에 서브쿼리는 메모리에 캐시되지 못한다.

```
explain
```

```
select * from 사원
```

```
where 사원번호 = (select round(rand()*100000000));
```

```
| 1 | PRIMARY          | 사원 | NULL | ALL | NULL | NULL | NULL |
| 2 | UNCACHABLE SUBQUERY | NULL | NULL | NULL | NULL | NULL | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+
rows | filtered | Extra |
+-----+-----+-----+
299157 | 100.00 | Using where |
NULL | NULL | No tables used |
+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

```
+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | poss |
+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | 사원 | NULL        | ALL | NULL |
+-----+-----+-----+-----+-----+-----+
```

select_type - MATERIALIZED

- IN절 구문에 연결된 서브쿼리가 임시 테이블을 생성한 뒤, 조인이나 가공 작업을 수행할 때 출력되는 유형
- IN절의 서브쿼리를 임시테이블로 만들어서 조인 작업을 수행하는 것

```
explain
```

```
select * from 사원
```

where 사원번호 in (select 사원번호 from 급여 where 시작일자 > '2020-01-01')

```
mysql> EXPLAIN
-> SELECT *
-> FROM 사원
-> WHERE 사원번호 IN (SELECT 사원번호 FROM 급여 WHERE 시작일자>'2020-01-01')
```

id	select_type	table	partitions	type	..	rows
1	SIMPLE	사원	NULL	ALL	..	299157
1	SIMPLE	<subquery2>	NULL	eq_ref	..	1
2	MATERIALIZED	급여	NULL	index	..	2838731

filtered	Extra
100.00	Using where
100.00	NULL

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	급여	NULL	index	PRIMARY	PRIMARY	7	NULL	2838398	3.46	Using where; Using index; LooseScan
1	SIMPLE	사원	NULL	eq_ref	PRIMARY	PRIMARY	4	tuning. 급여.사원번호	1	100.00	NULL

table

- 테이블명을 표시하는 항목, 테이블명이나 별칭을 출력
 - 서브쿼리나 임시테이블을 만들어서 별도의 작업을 수행할 때는 <subquery#> or <derived#>라고 출력됨

```
explain
SELECT 사원.사원번호, 급여.연봉
FROM 사원
```

```

JOIN (
    SELECT 사원번호, MAX(연봉) AS 연봉
    FROM 급여
    WHERE 사원번호 BETWEEN 10001 AND 20000
    GROUP BY 사원번호
) AS 급여
ON 사원.사원번호 = 급여.사원번호;

```

id	select_type	table	partitions	type	possible_keys	key
1	PRIMARY	<derived2>	NULL	system	NULL	NULL
1	PRIMARY	사원	NULL	const	PRIMARY	PRIMARY
2	DERIVED	급여	NULL	range	PRIMARY	PRIMARY

key_len	ref	rows	filtered	Extra
NULL	NULL	1	100.00	NULL
4	const	1	100.00	Using index
4	NULL	184756	100.00	Using where

3 rows in set (0.04 sec)

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL	184756	100.00	NULL
1	PRIMARY	사원	NULL	eq_ref	PRIMARY	PRIMARY	4	급여.사원번호	1	100.00	Using index
2	DERIVED	급여	NULL	range	PRIMARY, I_사용여부	PRIMARY	4	NULL	184756	100.00	Using where

3 rows in set, 1 warning (0.00 sec)

partitions

- 실행 계획의 부가정보로, 데이터가 저장된 논리적인 영역을 표시하는 항목
- 사전에 정의한 전체 파티션 중 특정 파티션에 선택적으로 접근하는 것이 SQL 성능적으로 유리,

너무 많은 영역의 파티션에 접근하는 것으로 출력된다면, 파티션 정의를 튜닝해야한다.

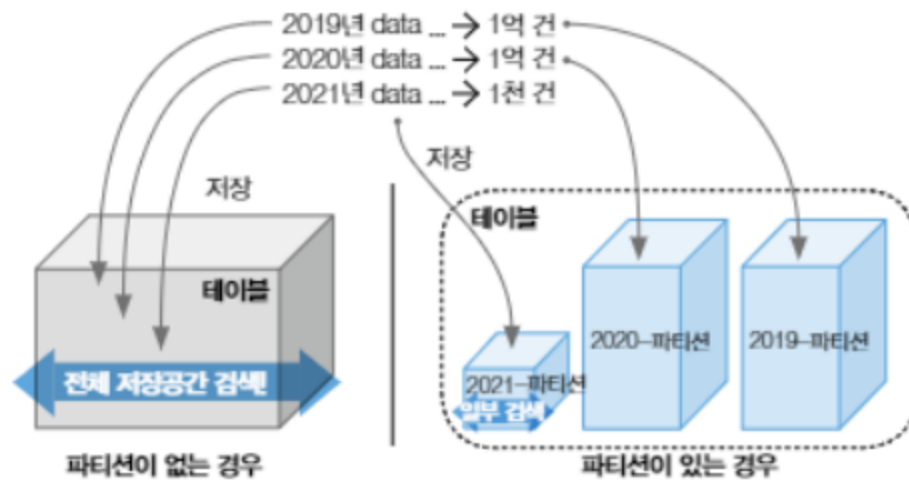


그림 3-39 파티션 유무에 따른 액세스 범위 비교

type

- 테이블의 데이터를 어떻게 찾을지에 대한 정보를 제공
- 처음부터 끝까지 전부를 확인할지, 인덱스를 통해 찾아갈지 등을 해석 가능

type - system

- 테이블에 데이터가 없거나, 한 개만 있는 경우로, 성능상 최상의 type

```

mysql> CREATE TABLE myisam_테이블 (
->   col1 INT(11) NULL DEFAULT NULL
-> )
-> ENGINE=MYISAM;
Query OK, 0 rows affected (0.004 sec)
mysql>
mysql> INSERT INTO myisam_테이블 VALUES(1);
Query OK, 1 row affected (0.002 sec)
mysql>
mysql> EXPLAIN
-> SELECT * FROM myisam_테이블;

```

```

+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table      | partitions | type   | possible_keys | key  |
+----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE      | myisam_테이블 | NULL       | system | NULL          | NULL |
+----+-----+-----+-----+-----+-----+-----+
key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+
NULL    | NULL | 1    | 100.00  | NULL |
+----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

```

type - const

- 조회되는 데이터가 단 1건일 때 출력되는 유형, 성능상 매우 유리
- 고유 인덱스나, 기본키를 사용하여 단 1건의 데이터에만 접근하면 되므로 매우 빠름

```

explain
select * from 사원
where 사원번호 = 10001;

```

id	select_type	table	partitions	type	possible_keys	key
1	SIMPLE	사원	NULL	const	PRIMARY	PRIMARY

key_len	ref	rows	filtered	Extra
4	const	1	100.00	NULL

1 row in set, 1 warning (0.00 sec)

```
mysql> explain
-> select * from 사원
-> where 사원번호 = 10001;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	사원	NULL	const	PRIMARY	PRIMARY	4	const	1	100.00	NULL

1 row in set, 1 warning (0.00 sec)

type - eq_ref

- 조인이 수행 될 때 드린본 테이블의 데이터에 접근하며, 고유인덱스 또는 기본키로 단 1건의 데이터를 조회하는 방식
- 드라이빙 테이블과 조인 키가 드리본 테이블에 유일하므로 조인이 수행될 때 가장 성능상 유리
- 두개의 실행계획 ID가 1로 동일하기에 조인함을 알 수 있음
 - 먼저 작성된 첫번째 부서사원_매핑 테이블이 드라이빙 테이블,
 - 이후 출력되는 두번째 행의 부서테이블이 드리본 테이블
 - type유형이 eq_ref로 출력됐으므로, 조인 수행시 기본 키나 고유 인덱스를 활용하여 1건의 데이터만 검색되는 과정을 미리 확인 가능

```
explain
select 매핑.사원번호, 부서.부서번호, 부서.부서명
  from 부서사원_매핑 as 매핑,
       부서
```

```
where 매핑.부서번호 = 부서.부서번호
and 매핑.사원번호 between 100001 and 10010;
```

```
+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | 매핑 | NULL | range | PRIMARY | PRIMARY |
| 1 | SIMPLE | 부서 | NULL | eq_ref | PRIMARY | PRIMARY |
+-----+-----+-----+-----+-----+-----+-----+
key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+
4 | NULL | 12 | 100.00 | Using where; Using index |
12 | 매핑.부서번호 | 1 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec))
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | no matching row in const table |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

type - index_merge

- 인덱스들이 동시에 사용되는 유형이다.
특정 테이블에 생성된 두 개이상 인덱스가 병합되어 동시에 적용됨
전문 인덱스는 제외

```
explain
select *
  from 사원
 where 사원번호 between 10001 and 10000
        and 입사일자 = '1985-11-21';
```

id	select_type	table	type	key	key_len	ref
1	SIMPLE	사원	index_merge	I_입사일자,PRIMARY	7,4	NULL

```
and 입사일자 = 1989-11-21 ;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	no matching row in const table

1 row in set, 1 warning (0.00 sec)

type - index

- 인덱스 풀 스캔을 의미
- 물리적인 인덱스 블록을 처음부터 끝까지 훑는 방식
 - 데이터를 스캔하는 대상이 인덱스라는 점이고, ALL과 비슷

```
EXPLAIN
SELECT 사원번호
FROM 직급
WHERE 직급명 = 'Manager';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	직급	NULL	index	PRIMARY	PRIMARY					

key_len	ref	rows	filtered	Extra
159	NULL	442961	10.00	Using where; Using index

1 row in set, 1 warning (0.00 sec)

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	직급	NULL	index	PRIMARY	PRIMARY	159	NULL	442129	10.00	Using where; Using index

1 row in set, 1 warning (0.00 sec)

type - index

- 테이블을 처음부터 끝까지 읽는 테이블 스캔 방식
 - 활용할 수 있는 인덱스가 없거나, 인덱스를 활용하는게 오히려 비효율적이라고 옵티마이저가 판단했을 때 선택됨
- ALL유형일때는 인덱스를 새로 추가하거나, 기존 인덱스를 변경하여 인덱스를 활용하는 방식으로 sql튜닝을 할 수 있으나,
전체 테이블중 10-20%분량의 데이터를 조회할 때는 오히려 all이 유리할수도

```
EXPLAIN
SELECT * FROM 사원;
```

id	select_type	table	partitions	type	possible_keys	key	key_len
1	SIMPLE	사원	NULL	ALL	NULL	NULL	NULL

ref	rows	filtered	Extra
NULL	299157	100.00	NULL

1 row in set, 1 warning (0.00 sec)

```
mysql> EXPLAIN
-> SELECT * FROM 사원;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	사원	NULL	ALL	NULL	NULL	NULL	NULL	297385	100.00	NULL

1 row in set, 1 warning (0.00 sec)

possile_keys

- 옵티마이저가 SQL문을 최적화 하고자 사용할 수 있는 인덱스 목록을 출력

- 실제 사용한 인덱스가 아닌, 사용할 수 있는 후보군의 기본 키와 인덱스 목록만 보여준다

key

- 옵티마이저가 SQL문을 최적화 하고자 사용한 기본키(PK) 또는 인덱스 명을 의미
→ 어느 인덱스로 데이터를 검색했는지 확인할 수 있으므로, 비효율적인 인덱스를 사용했거나, 인덱스 자체를 사용하지 않았다면 SQL튜닝의 대상이 된다.

```
EXPLAIN
SELECT 사원번호 FROM 직급
WHERE 직급명= 'Manager';
```

```
+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | 직급 | NULL        | index | PRIMARY        | PRIMARY |
+-----+-----+-----+-----+-----+-----+-----+
key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+
159      | NULL | 442961 | 10.00 | Using where; Using index |
+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

```
mysql> EXPLAIN
-> SELECT 사원번호 FROM 직급
-> WHERE 직급명= 'Manager';
+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | 직급 | NULL        | index | PRIMARY        | PRIMARY | 159      | NULL | 442129 | 10.00 | Using where; Using index |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

key_len

- 인덱스를 사용할 때는 인덱스 전체를 사용하거나, 일부만을 이용

- key_len은 사용한 인덱스의 바이트 수를 의미,
- UTF-8캐릭터셋 기준으로 INT는 4바이트, VARCHAR는 3바이트
- 예시
 - 직급 테이블의 데이터에 접근할 때 사원번호와 직급명, 시작일자를 결합한 기본키를 사용함
 - 실행계획의 key열은 primary, key_len은 159바이트
 - where절에서는 직급명을 사용하여 사원번호의 데이터를 조회중
- 사원번호 + 직급명 + 시작일자 열들로 구성된 pk를 어떻게 활용했는지 확인 가능
- 사원번호는 int → 4 바이트 , 직급명은 varchar → (50+1) * 3
- pk에서 사원번호 4바이트 , 직급명 155를 사용하여 key_len은 159

```
mysql> desc 직급;
```

Field	Type	Null	Key	Default	Extra
사원번호	int	NO	PRI	NULL	
직급명	varchar(50)	NO	PRI	NULL	
시작일자	date	NO	PRI	NULL	
종료일자	date	YES		NULL	

```
4 rows in set (0.00 sec)
```

```
explain
select 사원번호
from 직급
where 직급명 = 'Manager';
```

```
mysql> EXPLAIN
-> SELECT 사원번호
-> FROM 직급
-> WHERE 직급명 = 'Manager';
```

id	select_type	table	partitions	type	possible_keys	key
1	SIMPLE	직급	NULL	index	PRIMARY	PRIMARY

key_len	ref	rows	filtered	Extra
159	NULL	442961	10.00	Using where; Using index

1 row in set, 1 warning (0.00 sec)

```
mysql> explain
-> select 사원번호
-> from 직급
-> where 직급명 = 'Manager';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	직급	NULL	index	PRIMARY	PRIMARY	159	NULL	442129	10.00	Using where; Using index

1 row in set, 1 warning (0.00 sec)

ref

- 테이블 조인을 수행할 때 어떤 조건으로 해당 테이블에 액세스 했는지 알려주는 정보

```
explain
select 사원.사원번호, 직급.직급명
from 사원, 직급
where 사원.사원번호 = 직급.사원번호
and 사원.사원번호 between 10001 and 10100;
```

```
mysql> EXPLAIN
-> SELECT 사원.사원번호, 직급.직급명
-> FROM 사원, 직급
-> WHERE 사원.사원번호 = 직급.사원번호
-> AND 사원.사원번호 BETWEEN 10001 AND 10100;
```

id	select_type	table	partitions	type	possible_keys	key
1	SIMPLE	사원	NULL	range	PRIMARY	PRIMARY
1	SIMPLE	직급	NULL	ref	PRIMARY	PRIMARY

key_len	ref	rows	filtered	Extra
4	NULL	100	100.00	Using where; Using index
4	사원.사원번호	1	100.00	Using index

2 rows in set, 1 warning (0.00 sec)

```
mysql> explain
-> select 사원.사원번호, 직급.직급명
-> from 사원, 직급
-> where 사원.사원번호 = 직급.사원번호
-> and 사원.사원번호 between 10001 and 10100;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	직급	NULL	range	PRIMARY	PRIMARY	4	NULL	151	100.00	Using where; Using index
1	SIMPLE	사원	NULL	eq_ref	PRIMARY	PRIMARY	4	tuning. 직급.사원번호	1	100.00	Using index

2 rows in set, 1 warning (0.01 sec)

rows

- sql문을 수행하고자 접근하는 데이터의 모든 행수를 나타내는 예측 항목
- 디스크에서 데이터 파일을 읽고, 메모리에서 처리해야 할 행 수를 예상하는 값, 수치가 정확하진 않다.
- 최종 출력될 행 수가 아니라는 것을 유념

⇒ sql 문의 최종 결과 건수와 비교해 row수가 크게 차이가 날 때는 불필요하게 Mysql엔진까지 데이터를 많이 가져왔다는 뜻이므로, SQL튜닝 대상이 될 수 있다.

filtered

- SQL문을 통해 DB엔진으로 가져온 데이터 대상으로, 필터 조건에 따라 어느정도의 데이터를 제거했는지를 의미
- 단위는 퍼센트

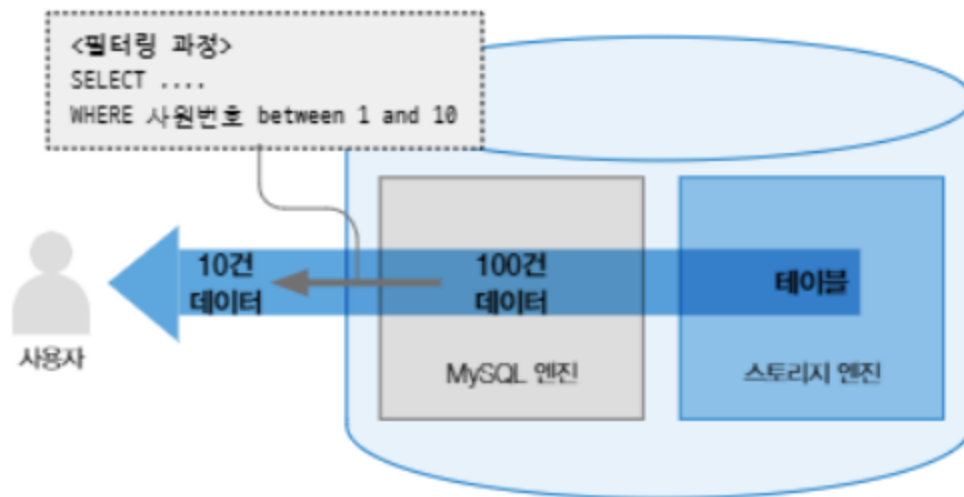


그림 3-40 filtered 항목 개념도

extra

- sql문을 어떻게 수행할 것인지에 대한 추가정보를 보여줌
- 세미콜론으로 구분하여 여러가지 정보를 나열할 수 있으며 30가지 항목으로 정리 가능

일반적이고, 현업에서 자주만나는예시

extra - Using temporary

- 데이터의 중간 결과를 제공하고자 임시 테이블을 생성하겠다는 의미
- 데이터를 가져와 저장한 뒤에 정렬 작업을 수행하거나 중복을 제거하는 작업을 사용

- distinct, group by, order by가 포함된 경우
- 임시 테이블을 메모리에 생성하나, 메모리 영역을 초과하여 디스크에 임시 테이블을 생성하면 성능저하의 원인이 될 수 있음
- 이 항목의 정보가 출력되는 쿼리는 SQL 튜닝의 대상이 될 수 있다.

extra - Using index

- 물리적인 데이터 파일을 읽지않고, 인덱스만을 읽어서 SQL문 요청 처리할 수 있는 경우, 커버링 인덱스라고 불린다.

```
EXPLAIN
SELECT 직급명
FROM 직급
WHERE 사원번호 = 100000;
```

```
mysql> EXPLAIN
-> SELECT 직급명
-> FROM 직급
-> WHERE 사원번호 = 100000;
```

id	select_type	table	partitions	type	possible_keys	key
1	SIMPLE	직급	NULL	ref	PRIMARY	PRIMARY

key_len	ref	rows	filtered	Extra
4	const	1	100.00	Using index

1 row in set, 1 warning (0.00 sec)

```
mysql> EXPLAIN
-> SELECT 직급명
-> FROM 직급
-> WHERE 사원번호 = 100000;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	직급	NULL	ref	PRIMARY	PRIMARY	4	const	1	100.00	Using index

1 row in set, 1 warning (0.01 sec)

extra - Using index

- 정렬이 필요한 데이터를 메모리에 올리고 정렬 작업을 수행한다는 의미
 - 이미 정렬된 인덱스를 사용하는 경우에는 해당 x
 - 인덱스를 사용하지 못할 경우, 정렬을 위해 메모리에 데이터를 올린다.
- SQL 튜닝 검토대상

extra - Using JOIN buffer

- 조인을 수행하기 위해 중간 데이터 결과를 저장하는 조인 버퍼를 사용한다는 의미
 - 드라이빙 테이블의 데이터에 먼저 접근한 결과를 조인버퍼에 담고 난 뒤, 조인 버퍼와 드리븐 테이블간에 서로 일치하는 조인 키 값을 찾는 과정을 수행

extra- Using index condition

- 필터조건을 스토리지 엔진으로 전달하여 필터링 작업에 대한 Mysql엔진의 부하를 줄이는 방식
- 스토리지 엔진의 데이터결과를 Mysql엔진으로 전송하는 데이터양을 줄여 성능 효율을 높일 수 있는 옵티마이저의 최적화 방식

extra- No exists

- 하나의 일치하는 행을 찾으면 ,추가로 행을 더 검색하지 않아도 될 때 출력되는 유형
 - 왼쪽외부조인 또는 오른쪽 외부조인에서 테이블에 존재하지 않는 데이터를 명시적으로 검사할 때 발생

3.2.3 종고 나뭇을 판단하는 기준

- SQL튜닝 대상을 검토할 때 참고할만한 기준
- 검토 대상인 실행계획 열은 select_type, type, extra

select_type 항목의 판단 기준



type 항목의 판단 기준



extra 항목의 판단 기준



3.2.4 확장된 실행 계획 수행

- 실행계획의 추가 정보를 확인하고자 한다면 DB에서 각각 지원하는 키워드를 확인

EXPLAIN FORMAT = TREE

- 트리형태로 추가 실행 계획 항목 확인 가능

```
EXPLAIN FORMAT = TREE
SELECT * FROM 사원
WHERE 사원번호 BETWEEN 100001 AND 20000;
```

```
mysql> EXPLAIN FORMAT = TREE
-> SELECT * FROM 사원
-> WHERE 사원번호 BETWEEN 100001 AND 20000;
+-----+
| EXPLAIN |
+-----+
| -> Zero rows (no matching row in const table) (cost=0..0 rows=0) |
| |
+-----+
1 row in set (0.01 sec)
```

EXPLAIN FORMAT = JSON

```
EXPLAIN FORMAT = JSON
SELECT * FROM 사원
WHERE 사원번호 BETWEEN 100001 AND 200000;
```

```
mysql> EXPLAIN FORMAT = JSON
-> SELECT * FROM 사원
-> WHERE 사원번호 BETWEEN 100001 AND 200000;
```

```
| EXPLAIN
```

```
| {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "4021.24"
    },
    "table": {
      "table_name": "사원",
      "access_type": "range",
      "possible_keys": [
        "PRIMARY"
      ],
      "key": "PRIMARY",
      "used_key_parts": [
        "사원번호"
      ],
      "key_length": "4",
      "rows_examined_per_scan": 20080,
      "rows_produced_per_join": 20080,
      "filtered": "100.00",
      "cost_info": {
        "read_cost": "2013.24",
        "eval_cost": "2008.00",
        "prefix_cost": "4021.24",
```

3.3 프로파일링

- sql문이 문제가 되는 병목 지점을 찾을 때 사용하는 수단

3.3.1 sql 프로파일링 실행하기

- mysql은 기본적으로 비활성화됨

```
show variables like 'profiling%';
```

```
mysql> show variables like 'profiling%';
```

Variable_name	Value
profiling	OFF
profiling_history_size	15

```
2 rows in set (0.001 sec)
```

```
mysql> show variables like 'profiling%';
```

Variable_name	Value
profiling	OFF
profiling_history_size	15

```
2 rows in set, 1 warning (0.01 sec)
```

- profiling on

```
set profiling = 'on';
```

```
mysql> set profiling = 'on';  
Query OK, 0 rows affected, 1 warning (0.00 sec)
```

- 특정 쿼리 profile 상세 내용 확인

```
SHOW PROFILE FOR QUERY 1;
```

3.3.2 프로파일링 결과 해석하기

표 3-1 일반적인 프로파일링 항목

항목	설명
starting	SQL 문 시작
checking permissions	필요 권한 확인
Opening tables	테이블을 열기
After opening tables	테이블을 연 이후
System lock	시스템 잠금
Table lock	테이블 잠금
init	초기화
optimizing	최적화
statistics	통계
preparing	준비
executing	실행
Sending data	데이터 보내기
end	끝
query end	질의 끝
closing tables	테이블 닫기
Unlocking tables	잠금 해제 테이블
freeing items	항목 해방
updating status	상태 업데이트
cleaning up	청소

- 프로파일링의 추가정보

- block I/O, CPU, SWAP 횟수 등에 대한 OS수준의 확장된 정보 제공

표 3-2 프로파일링의 선택 가능한 출력정보

옵션	설명
ALL	모든 정보를 표시
BLOCK IO	블록 입력 및 출력 작업의 횟수를 표시
CONTEXT SWITCHES	자발적 및 비자발적인 컨텍스트 스위치 수를 표시
CPU	사용자 및 시스템 CPU 사용 기간을 표시
IPC	보내고 받은 메시지의 수를 표시
PAGE FAULTS	주 페이지 오류 및 부 페이지 오류 수를 표시
SOURCE	함수가 발생하는 파일 이름과 행 번호와 함께 소스 코드의 함수 이름을 표시
SWAPS	스왑 카운트 표시

```
show profile all for query 1;
```

- sql 수행 단계별로 점유한 cpu량을 출력

```
SHOW PROFILE CPU FOR QUERY 1;
```

```
mysql> SHOW PROFILE CPU FOR QUERY 1;
+-----+-----+-----+-----+
| Status          | Duration | CPU_user | CPU_system |
+-----+-----+-----+-----+
| starting        | 0.000070 | 0.000000 | 0.000000   |
| freeing items   | 0.000069 | 0.000000 | 0.000000   |
| cleaning up     | 0.000007 | 0.000000 | 0.000000   |
+-----+-----+-----+-----+
3 rows in set, 1 warning (0.00 sec)
```

- 데이터 접근 단위인 블록 수준의 입력과 출력에 대한 정보를 추가로 출력

```
show profile block io for query 1;
```

```
mysql> show profile block io for query 1;
+-----+-----+-----+-----+
| Status          | Duration | Block_ops_in | Block_ops_out |
+-----+-----+-----+-----+
| starting        | 0.000070 | NULL         | NULL          |
| freeing items   | 0.000069 | NULL         | NULL          |
| cleaning up     | 0.000007 | NULL         | NULL          |
+-----+-----+-----+-----+
3 rows in set, 1 warning (0.00 sec)
```

표 3-3 확장된 프로파일링 항목

항목	설명
QUERY_ID	Query_ID
SEQ	동일한 QUERY_ID를 갖는 행의 표시 순서를 보여주는 일련번호
STATE	프로파일링 상태
DURATION	명령문이 현재 상태에 있었던 시간(초)
CPU_USER	사용자 CPU 사용량(초)
CPU_SYSTEM	시스템 CPU 사용량(초)
CONTEXT_VOLUNTARY	자발적 컨텍스트 전환의 수
CONTEXT_INVOLUNTARY	무의식적인 컨텍스트 전환의 수
BLOCK_OPS_IN	블록 입력 조작의 수
BLOCK_OPS_OUT	블록 출력 조작의 수
MESSAGES_SENT	전송된 통신 수

항목	설명
MESSAGES_RECEIVED	수신된 통신 수
PAGE_FAULTS_MAJOR	메이저 페이지 폴트의 수
PAGE_FAULTS_MINOR	마이너 페이지 폴트의 수
SWAPS	스왑 수
SOURCE_FUNCTION	프로파일링된 상태로 실행되는 소스 코드의 기능
SOURCE_FILE	프로파일링된 상태로 실행된 소스 코드의 파일
SOURCE_LINE	프로파일링된 상태로 실행된 소스 코드의 행